

Modeling and control of a quadcopter using code generation and FreeRTOS

1st Albin Svenske
MPSYS

Chalmers University of Technology
Gothenburg, Sweden
albinsve@chalmers.se

2nd Edvin Öhman
MPSYS

Chalmers University of Technology
Gothenburg, Sweden
edvinoh@chalmers.se

3rd Ananthakrishnan Kaimal
MPSYS

Chalmers University of Technology
Gothenburg, Sweden
kaimal@chalmers.se

4th Shubhajit Sahoo
MPSYS

Chalmers University of Technology
Gothenburg, Sweden
sahoos@chalmers.se

I. INTRODUCTION

This document handles the modeling and control of a quadcopter. This is done by fusing data from an accelerometer and a gyroscope with a complementary filter to get accurate measurements, modeling the dynamics of the quadcopter using Newtons law, then implementing this in the Mathworks tool Simulink using the Simscape language. C code is generated from the built Simulink model and run on the real quadcopter. C code is also written manually using the real time operating system FreeRTOS to run on the quadcopter for comparison of the two methods. As part of the control strategy, the goal is to stabilize and hover the drone mid-air by designing a Linear Quadratic Regulator(LQR) controller that can handle external disturbances. This is achieved through an iterative tuning process, allowing to refine the controller based on simulation and real-world performance. The project uses knowledge from the subjects of Logic Learning and decision, Modeling and Simulation and Linear control system design. The project ties together the knowledge gained from these courses into one project and implementing it on a real life system. Creating new systems using a model-based development approach is becoming more and more common due to its advantages in terms of time and money savings. The model-based development approach consists of using higher level languages such as Simulink and the Simscape language to build models of a system and then generate c code from these files to make the development easier and quicker, which is the main motivation for using the model-based development approach. The platform used for this project is the Bitcraze Crazyflie[1] but the development approach is applicable to any system.

II. ORIENTATION ESTIMATION

A. Axis estimation

In order to start modeling and controlling the quadcopter, the correct axis had to be determined as well as a filter to interpret the signals received by the two sensors on board

namely gyroscope and accelerometer. To achieve the desired X,Y and Z axis orientation, apply roll and pitch to the drone to understand the negative and positive effects to formulate the output based on the mathematical approach. Further, the logs from the Crazyflie user-interface were observed and this showed that moving the drone in the different directions and orientations could lead to the coordinate system similar to the following constellation:

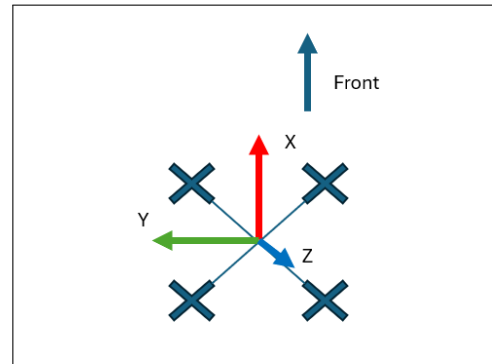


Fig. 1. Axis Of The Crazyflie Bit-craze Drone

In figure 1 it can be seen that the x-axis is in the front direction, the y-axis to the left if looking from above and the z-axis straight up. The sensor readings from the onboard gyroscope delivered the angular velocities of the drone in the respective axes which upon integrating over time could give the pitch (ϕ), roll (θ) and yaw (ψ) angles respectively.

B. Accelerometer readings

The onboard accelerometer measures the acceleration of the drone along its respective axes. The output is in the form of a 3D Vector $[f_x, f_y, f_z]$, where the values are in terms of g. When the drone is static, the output ideally reads $[x = 0, y = 0, z = 1]$ indicating the force being measured is gravity. Flipping the drone upside down means that the sign

of the z-reading changes. In more dynamic conditions, such as during free fall, the accelerometer can momentarily output values close to zero in all directions. This is because the drone is in a state of weightlessness experiencing no normal force, causing the sensed gravitational acceleration to drop to near zero. The accelerations does not drop to exactly zero due to disturbances and air resistance.

C. Complementary filter

To obtain accurate orientation estimation using the gyroscope and accelerometer, a filtering algorithm has to be designed. Each of the sensor performs well in different conditions i.e, the accelerometer provides better long-term stability with minimal steady-state errors, while the gyroscope performs more accurately during rapid movements and small accelerations.

The sensor readings that are being sent in to the filter are presented in the following images 2 and 3:

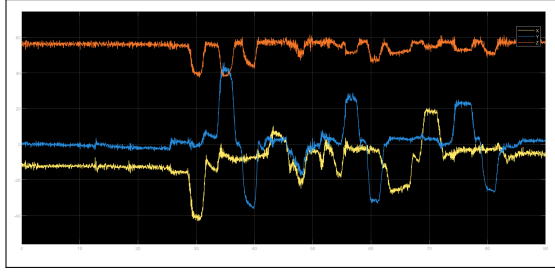


Fig. 2. Onboard Accelerometer Raw Data using Simulink

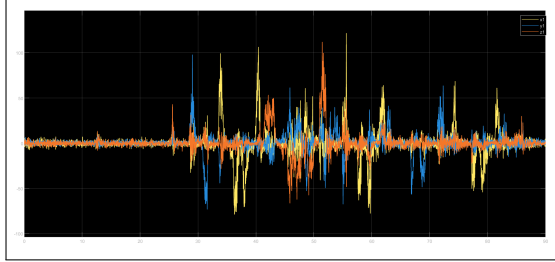


Fig. 3. Onboard Gyroscope Raw Data using Simulink

The structure of the complementary filter is shown in Figure 4:

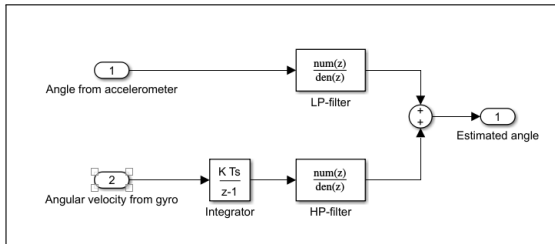


Fig. 4. Structure Of the Complementary Filter modeled using Simulink

With these transfer function equations in the Simulink model in figure 5 corresponds to the following continuous equation,

$$\Theta(s) = G(s)_{LP}\Theta_a + G(s)_{HP}\Theta_g \quad (1)$$

Where Θ is the estimated angle and Θ_a, Θ_g are the integrated angle estimations from the accelerometer and gyro respectively.

The implementation of the complementary filter, which effectively combines data from the drone's onboard gyroscope and accelerometer, is shown in Figure 5:

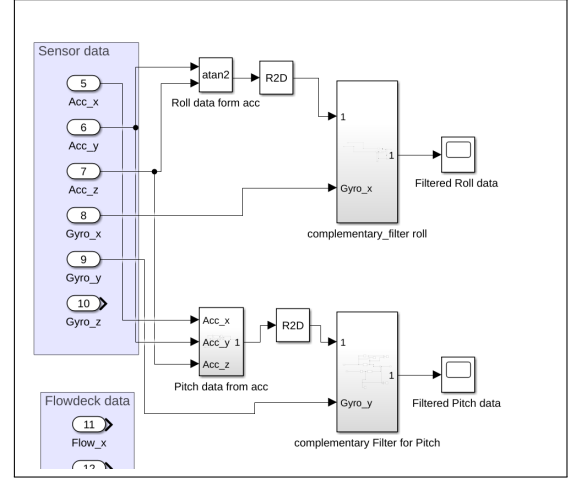


Fig. 5. Complementary Filter for generating Filtered Roll and Pitch data using Simulink

The inputs to the complimentary filter are converted to degrees from the gyroscope. Roll and pitch readings from the accelerometer are derived after some orientational calculations. The raw readings of acceleration, assumed that the drone is at rest can be described in force as:

$$\begin{bmatrix} {}^B f_x \\ {}^B f_y \\ {}^B f_z \end{bmatrix} = m {}^B R_w \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad (2)$$

The rotation matrix ${}^B R_w$ is defined with the following:

$$R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix} \quad (3)$$

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix} \quad (4)$$

$$R_z(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (5)$$

Multiplying them together gives the world to body rotation matrix (${}^B R_w = R_x R_y R_z$):

$${}^B R_w = \begin{bmatrix} c(\psi)c(\theta) & c(\psi)s(\theta)s(\phi) - s(\psi)c(\phi) & c(\psi)s(\theta)c(\phi) + s(\psi)s(\phi) \\ s(\psi)c(\theta) & s(\psi)s(\theta)s(\phi) + c(\psi)c(\phi) & s(\psi)s(\theta)c(\phi) - c(\psi)s(\phi) \\ -s(\theta) & c(\theta)s(\phi) & c(\theta)c(\phi) \end{bmatrix}$$

From these matrices and equation 2, roll and pitch from the accelerometer can be extracted as follows:

$$\varphi = \arctan\left(\frac{B f_y}{B f_z}\right) \quad (6)$$

$$\theta = \text{atan2}\left(-B f_x, \sqrt{B f_y^2 + B f_z^2}\right) \quad (7)$$

The yaw readings are not included in the complementary filter as it is difficult to estimate due to the gravitational forces that are not in play in this motion. The low-pass filter (LP – filter) on the accelerometer and high-pass filter (HP – filter) on the gyroscope are designed with the following transfer functions:

$$\text{LP – filter : } G(s)_{LP} = \frac{1}{\alpha s + 1} \quad (8)$$

$$\text{HP – filter : } G(s)_{HP} = \frac{\alpha s}{\alpha s + 1} \quad (9)$$

The LP and HP filters were discretized with the Backward Euler Method:

$$s \approx \frac{z - 1}{z T}, \quad T = 0.01 \quad (10)$$

This means that:

$$\text{LP – filter : } G(z)_{LP} = \frac{Tz}{(\alpha + T)z - \alpha}$$

$$\text{HP – filter : } G(z)_{HP} = \frac{\alpha z - \alpha}{(\alpha + T)z - \alpha}$$

The parameter α in the complementary filter controls the balance between two sensors on board. A higher α assigns more weight to the gyroscope data, allowing the filter to quickly respond to changes but risking drift accumulation over time. A lower α places more emphasis on the accelerometer data, which reduces drift but can introduce more noise and reduce responsiveness. The tuning process involves selecting the optimal α value to achieve the best trade-off between these effects. The tuning process aims to determine the optimal balance of how much information should be taken from each sensor. This balance is controlled by selecting an appropriate α value. After testing various values, a value of

$$\alpha = 0.7 \quad (11)$$

was chosen, as it combines the fast, short-term accuracy of the gyroscope with the stable, long-term reliability of the accelerometer, producing an estimate with improved stability.

Using the complementary filter enhances performance by leveraging the strengths of both sensors. Even with a sub-optimal α value, it is evident from the Figure 6 that the complementary filter significantly reduces the standing error in pitch when compared to the raw gyroscope readings.

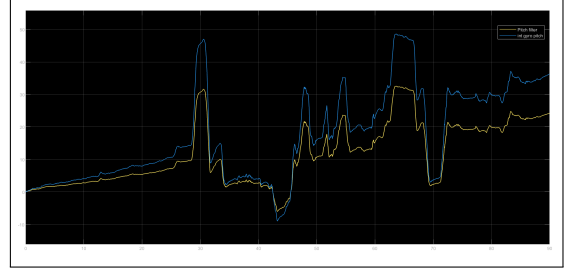


Fig. 6. Effect of Complementary Filter on Pitch Measurements using Simulink

A similar effect was observed when the same complementary filter was applied to the roll measurements. This observation is presented in Figure 7.

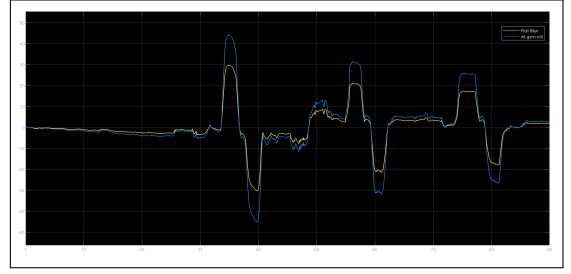


Fig. 7. Effect of Complementary Filter on Roll Measurements using Simulink

Later, to create a simple proportional controller (P-Controller) to control the system, a controller gain was applied to the error calculated between the reference and the measurement from the filter output. An appropriate value for this controller gain was:

$$P = 2000 \quad (12)$$

III. PLANT MODELING

The system was modeled using an acausal approach in Simscape, an equation-based tool that lets us define physical relationships directly without specifying signal flow direction. This approach mirrors real-world physics by describing components through fundamental equations, making modeling more intuitive and flexible. Simscape simplifies building complex multi-domain systems and enables easy modifications without redesigning computation order. Here, it has been used to accurately model the quad-copter's dynamics as a foundation for designing an effective controller.

A. Dynamics derivation

To design the controller for the quadcopter the dynamics are derived through newtons second law of motion. The drones global coordinates are defined as:

$$p = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \quad (13)$$

The velocity and acceleration have SI units, m/s and m/s² respectively and are defined as:

$$v = \dot{p} \quad (14)$$

$$a = \dot{v} \quad (15)$$

The rotors angular velocities are defined by the vector:

$$\omega_r = \begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \\ \omega_4 \end{bmatrix} \quad (16)$$

The thrust have SI unit, N and the corresponding thrust vector is then defined by:

$$T = b(\sum_{i=1}^4 \omega_i^2) \quad (17)$$

where T is the thrust and b is the lift constant. This gives the drones translational motion equation:

$$m\dot{v} = \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} - R_w^B \begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix} - Bv \quad (18)$$

Where B is the aerodynamic friction with SI unit, kg/s. The torque vector is defined as:

$$\Gamma = \begin{bmatrix} \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} \quad (19)$$

Where τ_{xyz} are the torque around each respective axis. With the cross configuration of the drone we get the torque equations by using figure 8:

$$\tau_x = \sin(\frac{\pi}{4})d(T_4 + T_3 - T_1 - T_2) = \frac{d}{\sqrt{2}}(T_4 + T_3 - T_1 - T_2) \quad (20)$$

$$\tau_y = \sin(\frac{\pi}{4})d(T_3 + T_2 - T_4 - T_1) = \frac{d}{\sqrt{2}}(T_3 + T_2 - T_4 - T_1) \quad (21)$$

$$\tau_z = k(T_1 + T_3 - T_2 - T_4) \quad (22)$$

where d is the distance from the rotor to the drone's center of mass and k is the drag constant with the SI units of m, N/(rad/s)², Nm/(rad/s)² respectively.

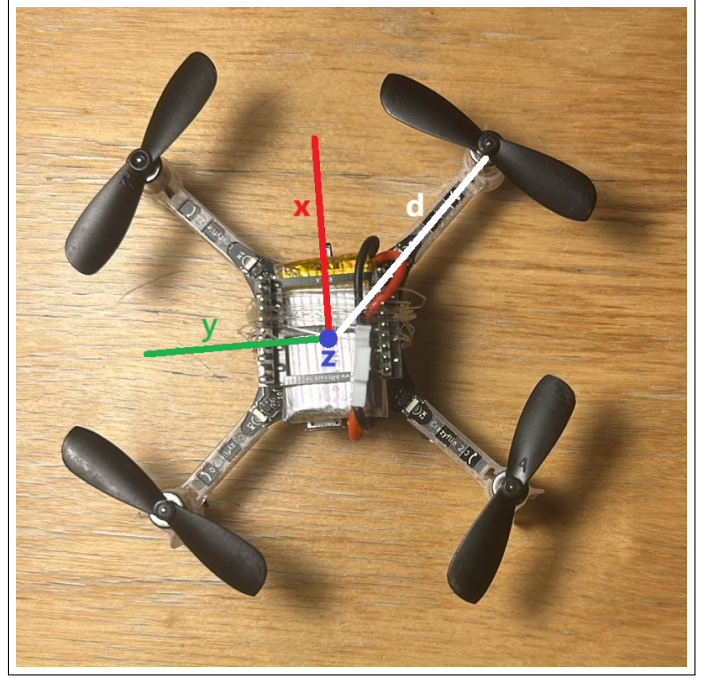


Fig. 8. View of the real drone with axis, motors and distance from the motors to the axis marked.

The rotational acceleration is given by the Euler equation:

$$J\dot{\omega} = -\omega \times J \cdot \omega + \Gamma \quad (23)$$

Where J is the inertia matrix. Now, the dynamics of the quadcopter are established. For clarity, they are summarized below.

$$J\dot{\omega} = -\omega \times J \cdot \omega + \Gamma \quad (24)$$

$$m\dot{v} = \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} - R_w^B \begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix} - Bv \quad (25)$$

$$\dot{\theta} = \omega \quad (26)$$

$$\dot{p} = v \quad (27)$$

$$\dot{v} = a \quad (28)$$

B. Simscape model

An open loop simulation in Simulink using Simscape was made to check whether there was any mistakes made in the equations. The tests were done by testing different inputs and checking if the position and angles gave reasonable responses. An example of the tests is shown in figure 9.

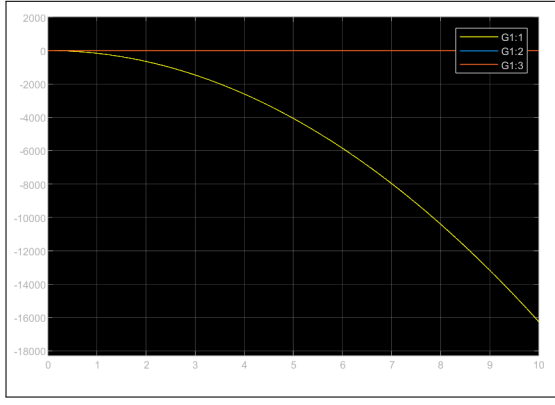


Fig. 9. Angle test by increasing motor 1 and 2 we should see a negative roll angle, which the plot shows.

IV. PLANT LINEARIZATION OF THE QUADROTOR

To enable control design for the quadrotor's attitude system, we begin by modeling its nonlinear rotational dynamics and then linearize the system around the hover operating point. This step is necessary because control strategies such as the Linear Quadratic (LQ) controller require linear models for effective application. The linearization process produces a continuous-time state-space representation that accurately describes the system's behavior near the equilibrium point. This linearized model can be expressed in the form $\Delta\dot{x} = A\Delta x + B\Delta u$, where $\Delta\dot{x}$ is the deviation of the state derivative from the operating point, x_0 , with x representing the state vector, u the input vector, and A and B the state and input matrices, respectively.

A. Obtaining the States and Inputs

The state vector Δx and input vector Δu are defined as:

$$\Delta x = \begin{bmatrix} \phi \\ \dot{\phi} \\ \theta \\ \dot{\theta} \\ \psi \\ \dot{\psi} \end{bmatrix}, \quad \Delta u = \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} \quad (29)$$

This state vector represents the roll, pitch, and yaw angular motion and their respective velocities.

B. Symbolic Linearization

The nonlinear model is symbolically linearized by computing Jacobian matrices of the system with respect to states and inputs. These are evaluated at the hover condition:

$$X_0 = 0, \quad U_0 = \left[\frac{mg}{4}, \frac{mg}{4}, \frac{mg}{4}, \frac{mg}{4} \right]^T \quad (30)$$

C. Linearized State-Space Representation

$$\underbrace{\begin{bmatrix} \dot{\phi} \\ \ddot{\phi} \\ \ddot{\theta} \\ \ddot{\psi} \end{bmatrix}}_{\Delta\dot{x}} = \underbrace{\begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}}_{A_{ex}} \underbrace{\begin{bmatrix} \phi \\ \dot{\phi} \\ \theta \\ \dot{\theta} \\ \psi \end{bmatrix}}_{\Delta x} + \underbrace{\begin{bmatrix} 0 & 0 & 0 & 0 \\ -2837.6 & -2837.6 & 2837.6 & 2837.6 \\ 0 & 0 & 0 & 0 \\ -1914.1 & 1914.1 & 1914.1 & -1914.1 \\ -428.2 & 428.2 & -428.2 & 428.2 \end{bmatrix}}_{B_{ex}} \underbrace{\begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix}}_{\Delta u}$$

D. Linearized State-Space Matrices

A matrix:

$$A_{ex} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (31)$$

B matrix:

$$B_{ex} = 10^3 \times \begin{bmatrix} 0 & 0 & 0 & 0 \\ -2.8376 & -2.8376 & 2.8376 & 2.8376 \\ 0 & 0 & 0 & 0 \\ -1.9141 & 1.9141 & 1.9141 & -1.9141 \\ 0.4282 & -0.4282 & 0.4282 & -0.4282 \end{bmatrix} \quad (32)$$

The matrix A_{ex} captures the local linear dynamics, while B_{ex} shows how control inputs (rotor thrusts) affect angular acceleration around each axis.

V. DESIGN OF LQR CONTROLLER

After substituting the numerical values of the constants, the resulting state-space representation matrices are given in Equations 32 and 33.

A. Discretizations of Matrix

We need to discretize the matrices, with the sampling time of 0.01 secs and using zero-order hold method we can get the following discretized matrices:

$$A_{\text{discrete}} = \begin{bmatrix} 1.0000 & 0.0100 & 0 & 0 & 0 \\ 0 & 1.0000 & 0 & 0 & 0 \\ 0 & 0 & 1.0000 & 0.0100 & 0 \\ 0 & 0 & 0 & 1.0000 & 0 \\ 0 & 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

$$B_{\text{discrete}} = \begin{bmatrix} -0.1419 & -0.1419 & 0.1419 & 0.1419 \\ -28.3756 & -28.3756 & 28.3756 & 28.3756 \\ 0.0957 & 0.0957 & 0.0957 & -0.0957 \\ -19.1414 & 19.1414 & 19.1414 & -19.1414 \\ -4.2821 & 4.2821 & -4.2821 & 4.2821 \end{bmatrix}$$

B. Calculations for the controller

After the linearization of the system around the equilibrium point $[0; 0; 0; 0; 0]$, the model is discretized with identity weighting matrices $Q = I$ and $R = I$, the optimal gain matrix K is computed by solving the Discrete-time Algebraic Riccati Equation (DARE):

$$P = A^T P A - A^T P B (R + B^T P B)^{-1} B^T P A + Q$$

resulting in the discrete-time state feedback control law:

$$u[k] = -Kx[k], \quad \text{where } K = (R + B^T P B)^{-1} B^T P A$$

The K matrix was then obtained from the dlqr command:
Using the MATLAB command

$$[K_d, S_d, P_d] = \text{dlqr}(A_d, B_d, Q, R)$$

Next, discrete-time LQR gain matrix K_d , where S_d is computed for finding the solution to the discrete-time algebraic Riccati equation and P_d is the closed-loop eigenvalue information. The K obtained for identity matrix Q and R is as follows :

$$K_{d1} = \begin{bmatrix} -0.0088 & -0.0089 & -0.0130 & -0.0131 & -0.0576 \\ -0.0088 & -0.0089 & 0.0130 & 0.0131 & 0.0576 \\ 0.0088 & 0.0089 & -0.0130 & -0.0131 & -0.0576 \\ 0.0088 & 0.0089 & 0.0130 & 0.0131 & 0.0576 \end{bmatrix}$$

C. Implementing the controller in the closed loop.

For the initial choice of Q and R as identity matrices, the resulting controller performance is shown below:

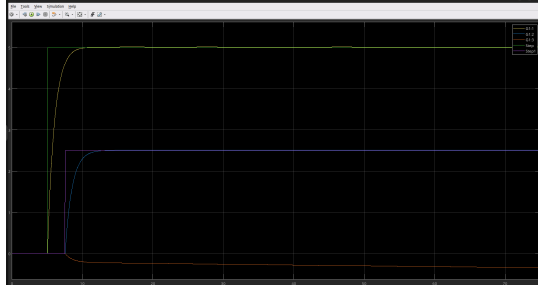


Fig. 10. State trajectories under the discrete LQR gain K_{d1} , using identity weighting matrices Q and R using Simulink.

The controller response is observed to be too sluggish for effective drone control in tracking the step input. To improve performance, the controller gain is enhanced by tuning the Q and R matrices. The updated Q and R matrices are as follows:

$$Q = \begin{bmatrix} 10000 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 10000 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad R = \begin{bmatrix} 0.5 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0.5 \end{bmatrix}$$

The resulting discrete-time LQR gain matrix K_{d2} is:

$$K_{d2} = \begin{bmatrix} -0.5873 & -0.0117 & -0.8705 & -0.0174 & -0.0580 \\ -0.5873 & -0.0117 & 0.8705 & 0.0174 & 0.0580 \\ 0.5873 & 0.0117 & 0.8705 & 0.0174 & -0.0580 \\ 0.5873 & 0.0117 & -0.8705 & -0.0174 & 0.0580 \end{bmatrix} \quad (33)$$

The corresponding system response to a step reference under this controller is illustrated below:

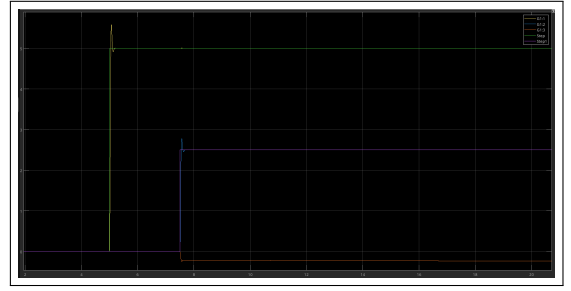


Fig. 11. State trajectories of the system under the discrete-time LQR gain K_{d2} , obtained using the tuned weighting matrices using Simulink.

The closed-loop eigenvalues can be used to validate the stability of the controller. The expression for closed loop eigen values is:

$$A_{cl} = A_d - B_d K_d$$

where A_{cl} is the closed-loop system matrix, A_d is the discrete-time system matrix, B_d is the discrete-time input matrix, and K_d is the discrete-time state feedback gain matrix. These are the eigen values of the same :

$$\text{eig}(A_{cl}) = \begin{bmatrix} 0.3333 \\ 0.3332 \\ 0.0002 \\ 0.0067 \\ 0.0005 \end{bmatrix}$$

Which shows the system is stable.

D. Tuning of the LQ parameters

The previously designed controller resulted in an overly aggressive response, which may not be suitable for smooth and stable drone operation. To mitigate this, the elements of the Q matrix were reduced to soften the control action. The updated weighting matrices are:

$$Q = \begin{bmatrix} 1000 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1000 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad R = \begin{bmatrix} 0.5 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0.5 \end{bmatrix}$$

The system response to a step reference under the resulting LQR controller is shown below:

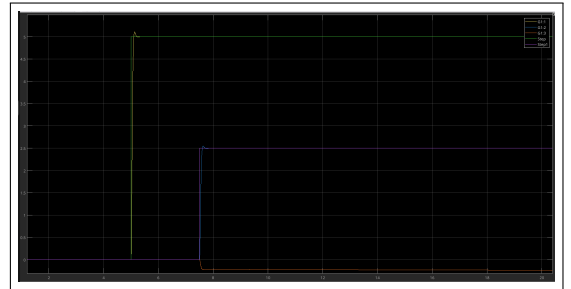


Fig. 12. State trajectories of the system using the discrete-time LQR gain K_{d2} , with the modified weighting matrices Q and R using Simulink.

VI. EVALUATION OF CONTROLLER DESIGN

The final LQR-based controller was tested in closed-loop Simulink simulations using a base thrust of 0.06, via complementary filters for roll and pitch estimation. The controller maintains the initial states. The Simulink model in Figure 13 illustrates the LQR controller implementation.

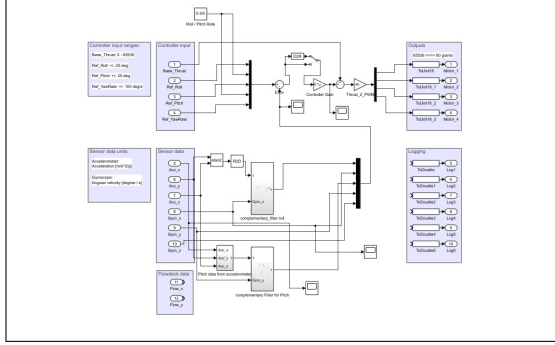


Fig. 13. LQR based Controller using Simulink

A. Behavior of roll and pitch disturbance and tracking

To assess the performance of the implemented LQ controller, a series of back tests were conducted involving pulse disturbance inputs for roll and pitch with a disturbance amplitude of 100 for 10 seconds and reference tracking was simulated for both roll and pitch dynamics once again. The outcomes are illustrated in the subsequent figures and analyzed based on the system's response.

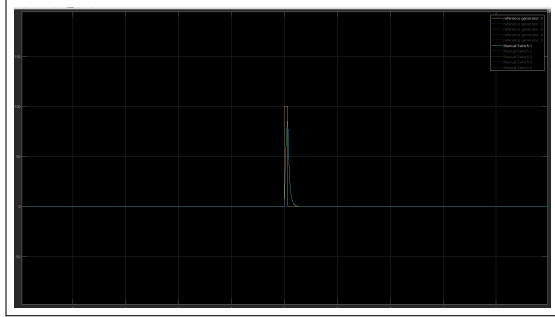


Fig. 14. LQR based Roll disturbance using Simulink

The system responds promptly and well damped demonstrating controllers robustness.

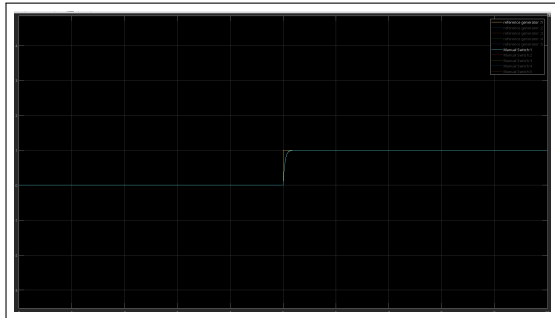


Fig. 15. LQR based Roll reference using Simulink

Figures 15 demonstrate the controller's strong tracking capability exhibiting a smooth and quick response and maintaining negligible steady-state error without any overshoot.

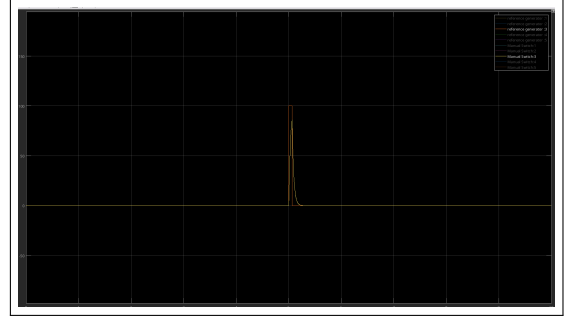


Fig. 16. LQR based Pitch disturbance using Simulink

Similar behavior is observed for roll disturbance.

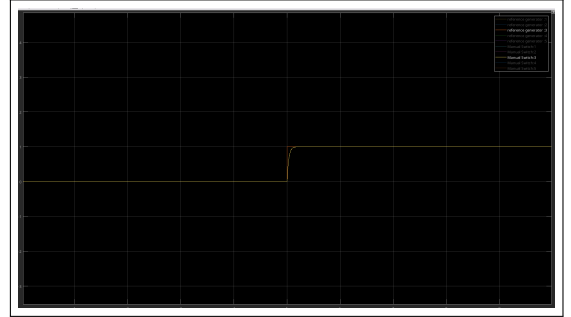


Fig. 17. LQR based Pitch reference using Simulink

For the ease of evaluation, the validation of pitch tracking is done independent of roll tracking.

B. Controller Behavior and Thrust

The system accepts reference commands for roll, pitch, and yaw rate, and computes the required thrust adjustments around a nominal base thrust using the gain matrix K_d . The outputs are converted into PWM signals sent to each motor.

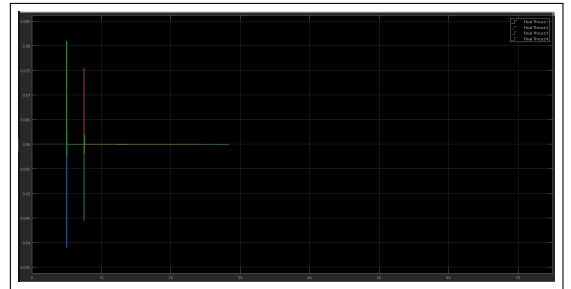


Fig. 18. Thrust values from the motors using Simulink

The plots derived from the simulation show the final thrust values for all four rotors in response to a reference command. The controller quickly adjusts the individual thrusts, with short-lived transients at the beginning of the simulation, before

all motor thrusts converge symmetrically around the base thrust of 0.06.

These observations confirm:

- The fast settling time of the LQR controller.
- Transient spikes are well-controlled, with no persistent oscillation or overshoot.
- The control effort is symmetrical, indicating balanced corrective actions for attitude stabilization.

C. Fault-Tolerance Analysis

To model the performance of the motor, test was performed where motor 2 and motor 4 had their maximum input signals restricted to 30,000. And for the same, the R matrix - input cost matrix was adjusted accordingly:

$$\begin{bmatrix} 0.01 & 0 & 0 & 0 \\ 0 & 1000 & 0 & 0 \\ 0 & 0 & 0.01 & 0 \\ 0 & 0 & 0 & 1000 \end{bmatrix}$$

The figures below display the result obtained when the inputs to motors 2 and 4 were penalized:

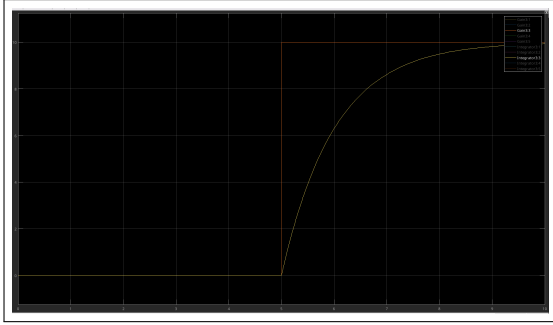


Fig. 19. Fault-tolerance Pitch reference tracking Motor 2 and 4 limited to 30,000 PWM angular constraints using Simulink

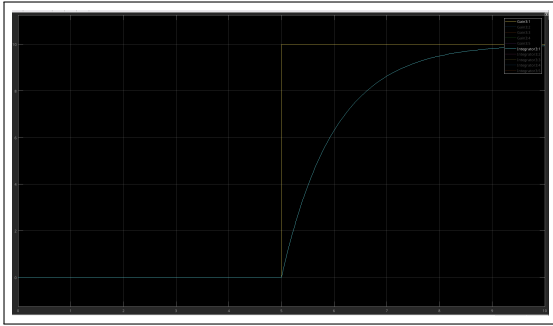


Fig. 20. Fault-tolerance Roll reference tracking Motor 2 and 4 limited to 30,000 PWM angular constraints using Simulink

When constraints were applied to the two motors, the controller showed a sudden increase in offset compensation. But these variations were minimal and largely insignificant, indicating that the controller maintains strong fault tolerance.

VII. IMPLEMENTATION USING A REAL TIME OPERATING SYSTEM

Beyond the implementation of C code using the C code generation feature from Simulink, another c code implementation was written using a real time operating system, in this case FreeRTOS. The use of a real time operating system is beneficial when dealing with different tasks that has to be finished within certain deadlines. For this project the tasks consists of calculating the complementary filters output and the controller output.

A. Developing the filter and control algorithms

The filter algorithm was implemented using a discretized version of equation 1 using the Euler backward method:

$$\hat{\Theta}_k = (1 - \gamma)\Theta_{a,k} + \gamma(\hat{\Theta}_{k-1} + h \cdot y_{g,k}) \quad (34)$$

where $\gamma = \frac{\alpha}{h+\alpha}$ and y_g is the gyro measurement. This was done for both roll and pitch estimation with the previous $\alpha = 0.7$ value.

The closed loop controller was created by simply first calculating the error based on the filter estimates and reference signals and then multiplying this with the previously calculated LQR gain in equation 33 to compute the output.

B. Using the tasks on the Crazyflie

Writing code manually for the Crazyflie can be done by dividing the filter and controller into two tasks. This is necessary to utilize the multiple threads available onboard. One problem with having two tasks within the same process is that they can read and write to the same variables in parallel with each other, resulting in calculation errors. To avoid this one can use the semaphore feature from FreeRTOS (see official documentation [2]). This means that each task can block other tasks from writing/reading to the same variables for the parts of the code that is necessary.

For this case the controller reads from the filter output and the sensor inputs, and the filter reads from the sensor inputs. When reading or writing to these states there is a semaphore blocking those respective variables, to avoid the problem explained in the previous paragraph. The control calculations can then be performed with the sensor and estimate semaphores being free for the filter task to use.

C. Simulation of the FreeRTOS implementation

To check the code for bugs and see if the performance was reasonable a simulation was made for the FreeRTOS implementation. This was done by running the implemented C code on a generated reference sequence for the roll angle. To illustrate the result from this simulation the reference data, filter estimation data and model true state was saved to a csv file that could then be plotted easily in MatLab. This can be seen in figures 21 and 22.

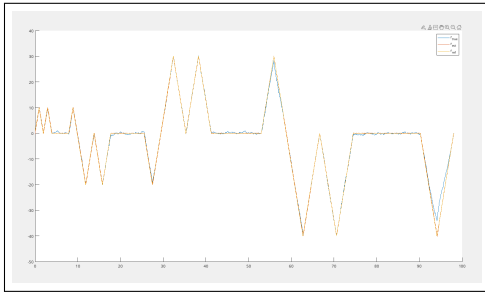


Fig. 21. Roll response upon RTOS implementation

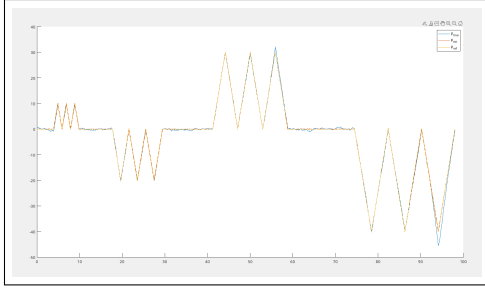


Fig. 22. Pitch response upon RTOS implementation

VIII. CONCLUSION

This project presented a comprehensive modeling and control framework for a quadrotor using a model-based approach. The nonlinear dynamics of the quadrotor were derived using the Newton–Euler method, capturing both translational and rotational motion. These dynamics were implemented in Simscape to verify correctness and simulate system behavior under various rotor inputs. Later, the system was linearized around the hover equilibrium point to obtain a state-space representation suitable for controller synthesis. The resulting linearized model enabled the design of a discrete-time LQR controller through symbolic Jacobian derivation and numerical discretizations using a zero-order hold method. Initial controller design using identity weighting matrices for Q and R yielded a stable but sluggish response. Subsequent tuning of the LQR cost matrices improved system responsiveness, resulting in better tracking performance. The final gain matrix was deployed in a closed-loop simulation, confirming stability and performance improvement. The entire workflow validates the effectiveness of model-based control design for real-time embedded applications on quadrotor, due to a lot of the work in the code generation being done automatically compared to manually writing the C code.

REFERENCES

- [1] Bitcraze AB, *Bitcraze: Home of the crazyflie nano quadcopter*, <https://www.bitcraze.io/>, Accessed: May 30, 2025.
- [2] FreeRTOS, *Freertos documentation*, Real Time Engineers Ltd., 2024. [Online]. Available: https://www.freertos.org/Documentation/RTOS_book.html.